

ZeroDrift — Truth Audit & Jury Q&A Pack

Every claim below was verified against the actual files on 5 Sept 2026. The one thing this audit could not verify is marked **NOT VERIFIED**.

1. Where every headline number comes from

Number	Verdict	Proof in the repo
1.27 s acquisition · 98.3 % lock	Computed by our software from a simulation run	<code>docs/media/telemetry_run.json</code> → summary block. Generated by <code>tools/export_telemetry.py</code> → <code>run_scenario()</code> in <code>src/fsoc_pat/runner.py</code> (same code path as the app). Scored against hidden ground truth in <code>src/fsoc_pat/metrics.py</code>
0 / 64 wrong-target locks	Computed — 64-run Monte-Carlo campaign	<code>runs/mc-leo/summary.json</code> → <code>n_runs: 64</code> , <code>runs_with_any_decoy_lock: 0</code> ; produced by <code>src/fsoc_pat/campaign.py</code>
99.3 % vs 14.7 % link availability	Computed, NOT hard-coded	<code>src/fsoc_pat/linkbudget.py</code> lines 130–131: <code>availability = (margin > 0).mean()</code> — fraction of frames where the computed optical margin is positive, fed by real tracking error. Written up in <code>docs/technical_report.md</code> §link budget
73 / 73 tests	Real — re-run by GitHub CI (both jobs green)	<code>tests/</code> + <code>.github/workflows/ci.yml</code>
Everything on the website	Read at runtime from the recordings — nothing typed in	<code>docs/demo.html</code> fetches <code>media/telemetry_*.json</code> ; tiles, charts, constants and even the beacon's position come from those files

2. The five honest questions

- Where did the simulation come from?** We wrote it — `src/fsoc_pat/` (~5,500 lines): `simulator.py` (sky, photon noise), `beacon.py` (trajectories incl. SGP4 on a real ISS TLE), `disturbance.py` (turbulence, vibration), `pipeline.py` (tracker), `control.py`. Not a downloaded framework.
- Where did the replay come from?** `tools/export_telemetry.py` ran the engine on three scenarios and logged every frame to `docs/media/telemetry_run.json` (+ turbulence, + decoys versions).
- Is the replay really our software's output?** Yes — same `run_scenario()` entry point as the desktop app, fixed seed 20261169; re-running the command reproduces identical numbers.
- Authentic vs simulated.** *Authentic external data*: the ISS orbital elements (TLE) and the published physics models (SGP4, link budget, turbulence — cited on slide 6). *Simulated*: the sky, camera, photons, disturbances — deliberately, that is what the problem statement asks for. *Computed*: every performance number.
- Can we say "measured"?** Yes — with one phrase attached: **"measured in our physics-based simulation against hidden ground truth."** Never imply a real optical link was measured.

NOT VERIFIED: whether the live Netlify site at this moment matches the repo (sandbox cannot reach netlify.app). 30-second self-check: tiles read 1.27 s / 98.3 % / 198 μrad, constants line reads FOV 6° · 30 fps · blink 4 Hz · slew limit 20°/s. Mismatch → redeploy `docs/`.

3. SAFE claims

- "Measured in our simulation against hidden ground truth."

- "Zero wrong-target locks in 64 randomised runs."
- "Every number regenerates from one command — the repo is public."
- "Open source, 73 automated tests passing."
- "AI benchmarked against the classical method: AUC 0.957 vs 0.900."
- "The algorithms are designed to transfer to real hardware — latency, mount limits and sensor noise are all modelled."

4. Claims to AVOID

- "Tested on real lasers / real hardware" ✗
- "99.3 % in the field" ✗ (it is a link-budget model on the simulated pass)
- "ISRO-validated" / "uses ISS signals" ✗ (we use its public orbit data only)
- "Our AI does the tracking" ✗ (the AI verifies; classical filters track)
- Any user-survey or market-testimonial numbers ✗ (we have none)

5. Fifteen likely jury questions — short answers

1. **Tested on real hardware?** No — the problem statement explicitly asks for validation *without* optical hardware; that is the deliverable. The algorithms are hardware-ready: mount limits, latency and sensor noise are all modelled.
2. **Will simulation transfer to reality?** The sim models the hard parts (40 ms latency, noise, turbulence). Validating PAT in simulation first is standard practice at ISRO/NASA.
3. **Where exactly is the AI?** A 497-parameter neural verifier that confirms each lock, trained on data from our own simulator. The gate stays classical; benchmarked 0.957 vs 0.900 AUC.
4. **Why not deep learning for detection?** Real-time at 30 fps on CPU; classical CFAR is tunable and explainable — AI only where it earns its place.
5. **What is the beacon?** The other terminal's identity light — in the sim, a moving point source on a real ISS orbit, blinking at 4 Hz.
6. **Why 4 Hz?** Camera runs 30 fps → Nyquist limit 15 Hz; 4 Hz gives ~7 samples per blink cycle for clean detection.
7. **What if it locks the wrong light?** The blink gate scores impostors near zero; 0 wrong locks in 64 runs; 1 stray frame in 3,900 in the decoy run, recovered automatically.
8. **What is a μ rad?** About 1 mm of sideways drift per km of distance. Our 198 μ rad median is deep inside the $\pm 3^\circ$ field.
9. **How do you handle the 40 ms delay?** Smith predictor — an internal model of the mount runs ahead, so commands aim where the target WILL be.
10. **What happens when the beacon blinks off?** COAST: the tracker points from the motion model's prediction. That is why detection duty is ~50 % but lock is 98.3 %.
11. **How do we know the numbers aren't faked?** The tracker never sees the answer; the simulator grades it afterwards. One command regenerates everything; the repository is public.
12. **Is the website live or a recording?** An honest replay of a logged run of the real engine — its banner says exactly that. The live version is the desktop app.
13. **Scalability?** The same engine already runs UAV, ground and decoy scenarios — each is one YAML file.
14. **Cost?** Zero: open source, no GPU, no cloud, any laptop.
15. **Future roadmap?** Hardware-in-the-loop with a real camera and mount, fine-steering-mirror integration, multi-terminal handover.

6. Competition & uniqueness

Likely rival approaches: generic OpenCV trackers (no identity — decoys kill them), MATLAB/Simulink models (not real-time, no app, licence cost), pure deep-learning trackers (GPU + dataset + unexplainable), and concept-only decks. Nobody at internal level typically brings all of: closed-loop 30 fps + identity-by-modulation + latency compensation + 64-run honest campaign + open repository + live replay site.

One line: *identity by blink, prediction past the delay, and every claim reproducible from one command.*

7. Housekeeping to remember

- National idea deadline $\approx$ 20 Sept — re-check the portal's PS-26169 team count before final submission.
- Golden rule: if a recording changes, regenerate the deck's chart and chips the same day. Deck and demo must never disagree.